

Complexity Theory: P, NP, Reductions & NP-Completeness

1 Problems: Optimization vs. Decision

Decision Problem

A decision problem returns only YES or NO. Complexity classes such as **P**, **NP**, and **NP-complete** are defined for decision problems.

Optimization version	Decision version
Find a minimum vertex cover.	Is there a vertex cover of size at most k ?
Find a maximum independent set.	Is there an independent set of size at least k ?
Find the largest clique.	Is there a clique of size at least k ?
Find the minimum TSP tour.	Is there a TSP tour of cost at most k ?

Threshold Rule

Minimum-type problems usually become “at most k ”; maximum-type problems usually become “at least k ”.

2 Polynomial Time and Class P

Polynomial Time

An algorithm is polynomial-time if its running time is bounded by

$$\mathcal{O}(n^c)$$

for some constant c , where n is the **encoded input length**.

Polynomial	Not polynomial
$\mathcal{O}(n)$, $\mathcal{O}(n \log n)$, $\mathcal{O}(n^2)$, $\mathcal{O}(n^5)$	$\mathcal{O}(2^n)$, $\mathcal{O}(n!)$

Why Polynomial Time Is the Standard Boundary

- It is stable across reasonable computation models under polynomial simulation.
- Polynomial bounds remain polynomial under composition, addition, and multiplication.
- It gives a robust theoretical notion of efficient solvability.

$$\mathcal{O}(n^a) + \mathcal{O}(n^b) = \mathcal{O}(n^{\max(a,b)}), \quad \mathcal{O}(n^a)\mathcal{O}(n^b) = \mathcal{O}(n^{a+b}).$$

Class P

$$\mathbf{P} = \{L \subseteq \{0,1\}^* : L \text{ is decidable in polynomial time}\}$$

using a deterministic algorithm/Turing machine.

Typical examples from the source notes include graph connectivity, shortest-path decision, minimum-spanning-tree decision, sorting/searching tasks, and primality testing.

3 Class NP: Certificates and Verification

Verifier Definition of NP

A language L is in **NP** if there is a polynomial-time verifier V and a polynomial p such that

$$x \in L \iff \exists y, |y| \leq p(|x|) \text{ and } V(x, y) = \text{YES.}$$

The string y is a **certificate** (or witness).

Solve vs. Verify

P: find the answer in polynomial time. **NP**: verify a proposed YES solution in polynomial time.

Problem	Certificate	Verification
SAT	truth assignment	substitute values and check the formula
CLIQUE	set of k vertices	check every pair has an edge
Vertex Cover	set of vertices	check size bound and cover every edge
Hamiltonian Cycle	ordering of vertices	check every vertex once, consecutive edges, and return edge

Why $P \subseteq NP$

If $L \in P$, use its polynomial-time decider as a verifier and ignore the certificate. Therefore

$$P \subseteq NP.$$

Name of NP

NP means **Nondeterministic Polynomial Time**, not “non-polynomial.” Membership in NP does not mean we know a polynomial-time solving algorithm.

4 P vs. NP, Complements, and co-NP

Open Question

$$P \stackrel{?}{=} NP$$

It is unknown whether every efficiently verifiable decision problem is also efficiently solvable.

Complement and co-NP

For a language L ,

$$\bar{L} = \{x \in \{0,1\}^* : x \notin L\}, \quad \text{coNP} = \{L : \bar{L} \in \text{NP}\}.$$

Known Relationships

$$P \subseteq NP, \quad P \subseteq \text{coNP}, \quad P \subseteq NP \cap \text{coNP}.$$

P is closed under complement: swap accept and reject in a polynomial-time decider.

It is also unknown whether

$$NP = \text{coNP}.$$

5 Polynomial-Time Reductions

Polynomial-Time Many-One Reduction

For languages $L_1, L_2 \subseteq \{0,1\}^*$,

$$L_1 \leq_p L_2$$

means there exists a polynomial-time computable function f such that

$$x \in L_1 \iff f(x) \in L_2.$$



How to Read $L_1 \leq_p L_2$

- L_1 can be translated to L_2 efficiently.
- A solver for L_2 gives a solver for L_1 .

- Therefore L_2 is at least as hard as L_1 .

Lemma 34.3

If

$$L_1 \leq_p L_2 \quad \text{and} \quad L_2 \in \mathbf{P},$$

then

$$L_1 \in \mathbf{P}.$$

Algorithm: compute $f(x)$ in polynomial time, solve $f(x)$ using the polynomial-time decider for L_2 , and return the same answer. Polynomial-time steps composed together remain polynomial.

Reduction Direction

To prove a target problem B is hard, reduce a known hard problem **to** B :

$$A \leq_p B.$$

Showing $B \leq_p A$ only shows that B is no harder than A .

6 NP-Hard and NP-Complete

NP-Hard

A problem H is **NP-hard** if

$$\forall L \in \mathbf{NP}, \quad L \leq_p H.$$

It does not have to be a decision problem and does not have to belong to **NP**.

NP-Complete

A decision problem C is **NP-complete** exactly when

$$C \in \mathbf{NP} \quad \text{and} \quad C \text{ is NP-hard.}$$

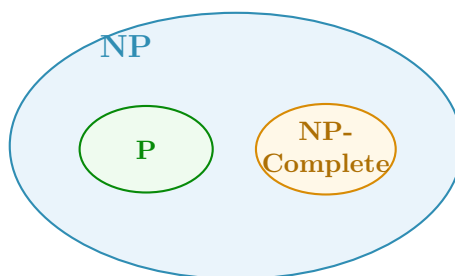
Equivalently, every language in **NP** polynomial-time reduces to C .

Class	Requirement	Meaning
P	solvable in polynomial time	efficiently solvable
NP	polynomial-size YES certificate verifiable in polynomial time	efficiently verifiable
NP-hard	every NP problem reduces to it	at least as hard as all of NP
NP-complete	in NP and NP-hard	hardest problems inside NP

Decision vs. Optimization Example

TSP decision: “Is there a tour of cost at most k ?” – **NP-complete** in the source notes.

TSP optimization: “Find the minimum-cost tour.” – **NP-hard** in the source notes.



NP-complete problems are inside NP; whether $P = NP$ is unknown.

7 Why NP-Completeness Matters

Theorem 34.4

If any **NP**-complete problem C is in **P**, then

$$\boxed{\mathbf{P} = \mathbf{NP}.}$$

Proof: for any $L \in \mathbf{NP}$, NP-completeness gives $L \leq_p C$. If $C \in \mathbf{P}$, Lemma 34.3 gives $L \in \mathbf{P}$. Hence $\mathbf{NP} \subseteq \mathbf{P}$. Together with $\mathbf{P} \subseteq \mathbf{NP}$, we get equality.

This is why one polynomial-time algorithm for one NP-complete problem would give polynomial-time algorithms for all problems in NP.

8 How to Prove a New Problem Is NP-Complete

Standard Proof Pattern

For a target decision problem B :

1. Show $B \in \mathbf{NP}$ by giving a polynomial-size certificate and polynomial-time verifier.
2. Choose a known NP-complete problem A .
3. Construct a polynomial-time mapping f proving

$$A \leq_p B.$$

4. Prove both directions:

$$x \in A \iff f(x) \in B.$$

5. Conclude B is NP-hard; with $B \in \mathbf{NP}$, conclude B is NP-complete.

Compact Proof Logic

$$\boxed{B \in \mathbf{NP}} + \boxed{A \leq_p B \text{ for known NPC } A} \implies \boxed{B \text{ is NP-complete.}}$$

9 Unified Reference Table

Term	Precise idea
Polynomial time	$\mathcal{O}(n^c)$ for constant c , measured in encoded input length.
Abstract problem	Mathematical relation between instances and solutions.
Encoding	Map from abstract instances to binary strings.
Concrete problem	Problem whose inputs are binary strings.
P	Decision problems solvable in polynomial time.
Certificate	Polynomial-size evidence for a YES instance.
Verifier	Polynomial-time algorithm checking (x, y) .
NP	Decision problems with polynomial-size YES certificates verifiable in polynomial time.
coNP	Languages whose complements belong to NP .
$L_1 \leq_p L_2$	Poly-time translation preserving YES/NO answers; L_2 is at least as hard.
NP-hard	Every problem in NP reduces to it.
NP-complete	In NP and NP-hard.

Core Relationships

$$\boxed{\mathbf{P} \subseteq \mathbf{NP}}, \quad \boxed{\mathbf{P} \subseteq \mathbf{coNP}}, \quad \boxed{\mathbf{NP-complete} = \mathbf{NP} \cap \mathbf{NP-hard} \text{ (for decision problems)}}$$

$$\boxed{L_1 \leq_p L_2 \ \& \ L_2 \in \mathbf{P} \Rightarrow L_1 \in \mathbf{P}}, \quad \boxed{\text{one NPC problem in } \mathbf{P} \Rightarrow \mathbf{P} = \mathbf{NP}.}$$

Important NP-Complete Problems

1 3-SAT Problem

Definition: 3-SAT

The **3-SAT problem** asks whether a Boolean formula in conjunctive normal form can be made true, where each clause has exactly three literals.

A 3-CNF formula has the form:

$$(C_1) \wedge (C_2) \wedge \cdots \wedge (C_m)$$

Each clause contains exactly three literals.

Example

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$$

Question:

Is there a truth assignment that makes the formula true?

Checking a Certificate

A certificate is a truth assignment such as:

$$x_1 = T, \quad x_2 = T, \quad x_3 = F, \quad x_4 = T$$

We can check every clause in polynomial time. Therefore, 3-SAT is in **NP**.

2 Vertex Cover

Definition: Vertex Cover

Given an undirected graph $G = (V, E)$, a **vertex cover** is a subset $C \subseteq V$ such that every edge in E has at least one endpoint in C .

Decision Version

Given a graph G and an integer k , determine whether G has a vertex cover of size at most k .

$$|C| \leq k$$

Example

Let:

$$V = \{1, 2, 3, 4\}, \quad E = \{(1, 2), (2, 3), (3, 4)\}$$

A vertex cover is:

$$C = \{2, 3\}$$

Because every edge touches vertex 2 or vertex 3.

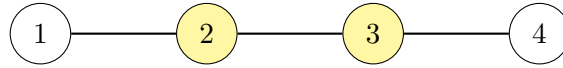


Figure: Vertices 2 and 3 cover all edges.

3 Independent Set

Definition: Independent Set

Given an undirected graph $G = (V, E)$, an **independent set** is a subset $I \subseteq V$ such that no two vertices in I are adjacent.

Decision Version

Given a graph G and an integer k , determine whether G has an independent set of size at least k .

$$|I| \geq k$$

Example

For:

$$V = \{1, 2, 3, 4\}, \quad E = \{(1, 2), (2, 3), (3, 4)\}$$

A valid independent set is:

$$I = \{1, 3\}$$

There is no edge between 1 and 3.

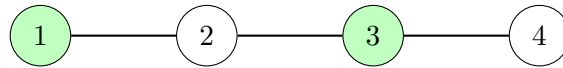


Figure: Vertices 1 and 3 form an independent set.

4 Clique

Definition: Clique

Given an undirected graph $G = (V, E)$, a **clique** is a subset $K \subseteq V$ such that every pair of distinct vertices in K is connected by an edge.

Decision Version

Given a graph G and an integer k , determine whether G contains a clique of size at least k .

$$|K| \geq k$$

Example

Let:

$$V = \{1, 2, 3, 4\}$$

$$E = \{(1, 2), (1, 3), (2, 3), (3, 4)\}$$

Then:

$$K = \{1, 2, 3\}$$

is a clique because every pair among 1, 2, and 3 is connected.

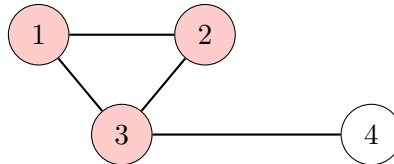


Figure: $\{1, 2, 3\}$ is a clique.

5 Set Cover

Definition: Set Cover

Given a universe:

$$U = \{u_1, u_2, \dots, u_n\}$$

and a collection of subsets:

$$\mathcal{S} = \{S_1, S_2, \dots, S_m\}$$

a **set cover** is a subcollection $\mathcal{C} \subseteq \mathcal{S}$ such that every element of U belongs to at least one set in $\mathcal{C}$.

Decision Version

Given U , $\mathcal{S}$, and integer k , determine whether there exists a set cover of size at most k .

$$|\mathcal{C}| \leq k$$

Example

Let:

$$U = \{1, 2, 3, 4, 5\}$$

$$S_1 = \{1, 2, 3\}, \quad S_2 = \{2, 4\}, \quad S_3 = \{3, 4, 5\}$$

Then:

$$\{S_1, S_3\}$$

is a set cover because:

$$S_1 \cup S_3 = \{1, 2, 3, 4, 5\} = U$$

NP-Complete Proof Chain

Compact Actual Proofs - Exam Ready

Proof Format to Use

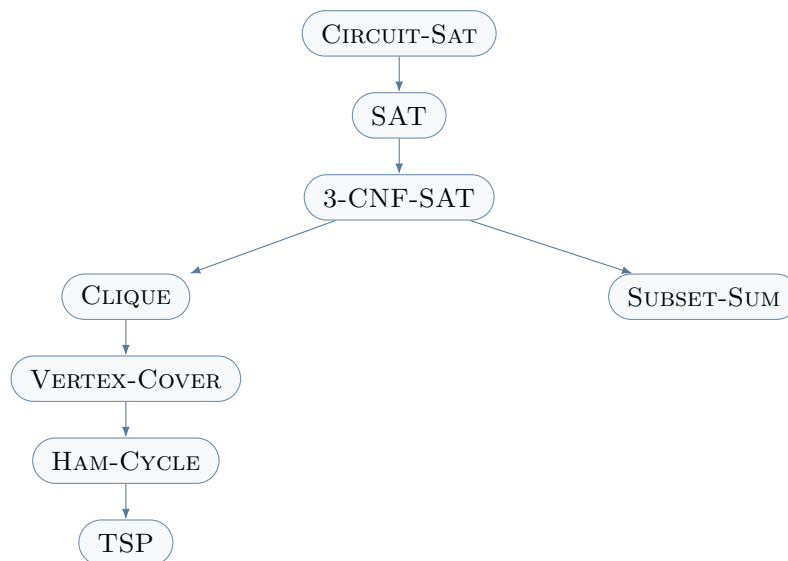
For every proof, write only these six parts:

In NP $\rightarrow$ Build $\rightarrow$ $\Rightarrow$ $\rightarrow$ $\Leftarrow$ $\rightarrow$ Time $\rightarrow$ Conclusion

Reduction Chain

$\text{CIRCUIT-SAT} \leq_p \text{SAT} \leq_p \text{3-CNF-SAT} \leq_p \text{CLIQUE} \leq_p \text{VERTEX-COVER} \leq_p \text{HAM-CYCLE} \leq_p \text{TSP}$

$\text{3-CNF-SAT} \leq_p \text{SUBSET-SUM}$.



1 Circuit-SAT is NP-Complete

Problem

Does a Boolean circuit have some input assignment that makes its output 1?

Actual Proof

In NP. Certificate = input bits. Evaluate each gate once. For g gates, verification is $O(g)$.

Build. Let L be any problem in NP with polynomial-time verifier $V(x, y)$. Fix input x . Build a Boolean circuit C_x whose free inputs are the certificate bits y and whose gates simulate the computation of $V(x, y)$.

$\Rightarrow$. If $x \in L$, some certificate y makes $V(x, y)$ accept. Using that y as circuit input makes $C_x = 1$.

$\Leftarrow$. If some input y makes $C_x = 1$, then the simulated verifier accepts (x, y) . Hence $x \in L$.

Time. V uses polynomially many steps, so the simulation circuit has polynomially many gates and is built in polynomial time.

Conclusion. Every problem in NP reduces to Circuit-SAT, and Circuit-SAT is in NP. Therefore

CIRCUIT-SAT is NP-complete.

2 SAT is NP-Complete

Problem

Does a Boolean formula have a satisfying truth assignment?

Reduction

$$\text{CIRCUIT-SAT} \leq_p \text{SAT}$$

Actual Proof

In NP. Certificate = truth assignment. Substitute values and evaluate the formula in polynomial time.

Build. From a circuit C , create one variable for every wire. For each gate add a constraint:

$$\begin{aligned} z = x \wedge y & : z \leftrightarrow (x \wedge y), \\ z = x \vee y & : z \leftrightarrow (x \vee y), \\ z = \neg x & : z \leftrightarrow \neg x. \end{aligned}$$

Conjoin all gate constraints and require the output variable o to be true:

$$F = (\text{all gate constraints}) \wedge o.$$

$\Rightarrow$. If C outputs 1 for some input, assign each wire-variable its actual circuit value. Every constraint is true and $o = 1$, so F is satisfiable.

$\Leftarrow$. If F is satisfiable, all wire values obey the gate rules and $o = 1$. Thus the original circuit has an input making output 1.

Time. One constant-size condition per gate, so $|F| = O(|C|)$.

Conclusion. Circuit-SAT $\leq_p$ SAT and SAT $\in$ NP, so

$$\text{SAT is NP-complete.}$$

3 3-CNF-SAT is NP-Complete

Problem

Does a CNF formula with exactly three literals per clause have a satisfying assignment?

Reduction

$$\text{SAT} \leq_p \text{3-CNF-SAT}$$

Actual Proof

In NP. Guess an assignment and check every clause.

Build. Introduce a helper variable for each subformula. Enforce each operation by clauses:

$$\begin{aligned} z = x \wedge y & : (\neg z \vee x)(\neg z \vee y)(z \vee \neg x \vee \neg y), \\ z = x \vee y & : (z \vee \neg x)(z \vee \neg y)(\neg z \vee x \vee y), \\ z = \neg x & : (z \vee x)(\neg z \vee \neg x). \end{aligned}$$

Force the root variable to be true. Convert shorter clauses to 3-literal clauses:

$$\begin{aligned} (a \vee b) & \rightsquigarrow (a \vee b \vee p)(a \vee b \vee \neg p), \\ (a) & \rightsquigarrow (a \vee p \vee q)(a \vee p \vee \neg q)(a \vee \neg p \vee q)(a \vee \neg p \vee \neg q). \end{aligned}$$

$\Rightarrow$. A satisfying assignment of the original formula extends to helper variables using the true values of their subformulas, so every new clause is satisfied.

$\Leftarrow$. Any satisfying assignment of the new formula forces each helper variable to represent the correct operation; since the root is true, the original formula is true.

Time. Each original operation creates only $O(1)$ variables/clauses.

Conclusion.

3-CNF-SAT is NP-complete.

4 Clique is NP-Complete

Problem

Does a graph contain k vertices that are all pairwise adjacent?

Reduction

3-CNF-SAT $\leq_p$ CLIQUE

For m clauses, ask for a clique of size m .

Actual Proof

In NP. Certificate = k vertices. Check all pairs for edges.

Build. Let $\phi = C_1 \wedge \dots \wedge C_m$. Create one vertex for each literal occurrence. Add an edge between two vertices iff:

- they come from different clauses, and
- they are not contradictory (x vs. $\neg x$).

Set $k = m$.

$\Rightarrow$. If ϕ is satisfiable, choose one true literal from each clause. The chosen literals never contradict, so all chosen vertices are connected. Hence there is an m -clique.

$\Leftarrow$. An m -clique can contain at most one vertex from each clause, so it contains exactly one from every clause. No two selected literals contradict. Make all selected literals true; then every clause is satisfied.

Time. $3m$ vertices and at most $O(m^2)$ edges.

Conclusion.

CLIQUE is NP-complete.

5 Vertex-Cover is NP-Complete

Problem

Can at most k vertices cover every edge of a graph?

Reduction

CLIQUE $\leq_p$ VERTEX-COVER

Map (G, k) to $(\overline{G}, |V| - k)$.

Actual Proof

In NP. Certificate = set C of at most k vertices. Check every edge has an endpoint in C .

Build. Construct the complement graph $\overline{G}$: two distinct vertices are adjacent in $\overline{G}$ exactly when they are not adjacent in G .

$\Rightarrow$. If S is a k -clique in G , then there is no edge of $\overline{G}$ inside S . Therefore every edge of $\overline{G}$ touches $V - S$, so $V - S$ is a vertex cover of size $|V| - k$.

$\Leftarrow$. If C is a vertex cover of $\overline{G}$ with $|C| \leq |V| - k$, then at least k vertices are outside C . No edge of $\overline{G}$ joins two outside vertices, so every pair of them is adjacent in G . They contain a clique of size k .

Time. Constructing $\overline{G}$ takes $O(|V|^2)$.

Conclusion.

VERTEX-COVER is NP-complete.

6 Hamiltonian Cycle is NP-Complete

Problem

Does a graph contain a cycle visiting every vertex exactly once?

Reduction

VERTEX-COVER $\leq_p$ HAM-CYCLE

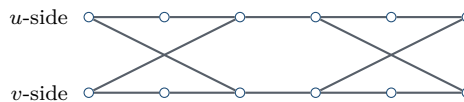
Replace each original edge by the standard edge gadget; connect gadget sides into one chain per original vertex; add k selector vertices.

In NP

Certificate = an ordering of all vertices. Check uniqueness, every consecutive edge, and the final edge back to the start.

Edge Gadget

For each original edge $\{u, v\}$ create a 12-vertex gadget with a u -side and a v -side:



Only Gadget Fact Needed

A Hamiltonian traversal of this gadget can represent exactly these useful cases:

u chosen v chosen u and v both chosen.

Thus the gadget can be fully visited iff at least one endpoint of the original edge is selected.

Actual Correctness Proof

Build. Join all u -sides of gadgets incident to each original vertex u into one long u -chain. Add selector vertices $s_1, \dots, s_k$ and connect each selector to the two ends of every vertex-chain.

$\Rightarrow$. Let C be a vertex cover with $|C| \leq k$ (pad to exactly k if needed). Use selectors to connect the chains of vertices in C . For every original edge $\{u, v\}$, at least one endpoint is in C , so use the gadget traversal corresponding to u , v , or both. Hence every gadget vertex is visited exactly once, producing a Hamiltonian cycle.

$\Leftarrow$. Given a Hamiltonian cycle, remove the k selector vertices. The cycle splits into at most k paths, each following one original vertex-chain. Call those original vertices selected. Every edge gadget must be fully visited, and by the gadget fact this requires at least one endpoint-chain of that edge to be selected. Therefore the selected vertices cover every original edge, so they form a vertex cover of size at most k .

Time. Each edge gives 12 vertices; all chain and selector connections are polynomial in $|V| + |E|$.

Conclusion.

HAM-CYCLE is NP-complete.

7 Traveling-Salesman Problem is NP-Complete

ProblemIs there a tour visiting every vertex once, returning to the start, with cost at most B ?**Reduction**HAM-CYCLE $\leq_p$ TSPFor $n = |V|$, make a complete graph with cost 1 on original edges, cost 2 on missing edges, and set $B = n$.**Actual Proof****In NP.** Certificate = tour order. Check all vertices once and sum the n edge costs.**Build.**

$$c(u, v) = \begin{cases} 1, & \{u, v\} \in E, \\ 2, & \{u, v\} \notin E. \end{cases} \quad B = n.$$

 $\Rightarrow$. A Hamiltonian cycle in G uses n original edges, each of cost 1, so the same tour in G' has cost n . $\Leftarrow$. Any TSP tour uses exactly n edges. Since every edge costs at least 1, total cost $\leq n$ is possible only if every used edge costs 1. Therefore all tour edges were original edges of G , so the tour is a Hamiltonian cycle.**Time.** Assigning costs to all $O(n^2)$ pairs is polynomial.**Conclusion.**

TSP is NP-complete.

8 Subset-Sum is NP-Complete

ProblemGiven positive integers and target t , is there a subset whose sum is exactly t ?**Reduction**3-CNF-SAT $\leq_p$ SUBSET-SUM

Use decimal columns: one per variable and one per clause.

Actual Proof**In NP.** Certificate = chosen numbers. Add them and compare with t .**Build.** For variables $x_1, \dots, x_n$ and clauses $C_1, \dots, C_m$, use $n + m$ decimal columns.

- For each x_i , create T_i and F_i . Both have digit 1 in variable column i .
- In clause column j , T_i has 1 if $x_i \in C_j$; F_i has 1 if $\neg x_i \in C_j$.
- For each clause add two slack numbers with digits 1 and 2 in that clause column.
- Target digits: variable columns = 1; clause columns = 4.

No carry occurs because every column sum is below 10.

 $\Rightarrow$. From a satisfying assignment choose exactly one of T_i, F_i per variable. Each variable column

becomes 1. A satisfied clause receives 1, 2, or 3 from true literals; add slack to reach 4:

$$1 \rightarrow 1 + 2, \quad 2 \rightarrow 2, \quad 3 \rightarrow 1.$$

So the chosen numbers sum to the target.

$\Leftarrow$. If a subset sums to the target, each variable column equals 1, so exactly one of T_i, F_i is chosen; this defines a truth assignment. In a clause column, slack contributes at most 3 but the target is 4, so at least one chosen variable number contributes 1. Hence that clause has a true literal. Therefore all clauses are satisfied.

Time. We create $2n + 2m$ numbers, each with $n + m$ digits, so the construction is polynomial.

Conclusion.

SUBSET-SUM is NP-complete.

Graph NP-Completeness Proofs

Compact Actual Proofs - Exam Ready

Proof Format to Use

For every NP-completeness proof, keep this order:

$\boxed{\text{In NP}} \rightarrow \boxed{\text{Build}} \rightarrow \boxed{\Rightarrow} \rightarrow \boxed{\Leftarrow} \rightarrow \boxed{\text{Time}} \rightarrow \boxed{\text{Conclusion}}$

Reduction Map

$\text{CLIQUE} \leq_p \text{INDEPENDENT-SET}$, $3\text{-COLOR} \leq_p \text{VERTEX-COLOR}$, $\text{VERTEX-COLOR} \leq_p \text{CLIQUE-PARTITION}$

Also: $3\text{-CNF-SAT} \leq_p 3\text{-COLOR}$. For edge coloring, use Holyer's theorem.

1 Independent Set is NP-Complete

Problem

Does graph G contain at least k vertices with no edge between any two of them?

Reduction

$\boxed{\text{CLIQUE} \leq_p \text{INDEPENDENT-SET}}$

Given (G, k) , build the complement graph $\overline{G}$ and keep the same k .

Actual Proof

In NP. Certificate = k vertices. Check every chosen pair; none may be adjacent.

Build. In $\overline{G}$, two distinct vertices are adjacent exactly when they are *not* adjacent in G .

$\Rightarrow$. If S is a k -clique in G , every pair in S is an edge of G . Hence no pair is an edge of $\overline{G}$. So S is an independent set in $\overline{G}$.

$\Leftarrow$. If S is an independent set of size k in $\overline{G}$, no pair in S is an edge of $\overline{G}$. Therefore every pair is an edge of G , so S is a k -clique in G .

Time. Build $\overline{G}$ by checking all vertex pairs: $O(|V|^2)$.

Conclusion.

$\boxed{\text{INDEPENDENT-SET is NP-complete.}}$

2 Vertex Coloring is NP-Complete

Problem

Can the vertices of G be colored with at most k colors so that adjacent vertices have different colors?

Hard Core

$\boxed{3\text{-CNF-SAT} \leq_p 3\text{-COLOR}}$

Then 3-COLOR is the special case $k = 3$ of VERTEX-COLOR.

Actual Proof

In NP. Certificate = one color per vertex. Check every edge; its endpoints must have different colors.

Build. From a 3-CNF formula:

- Create special vertices T, F, B and connect them as a triangle. They must get three different colors.
- For each variable x_i , create x_i and $\neg x_i$; connect them to each other and to B . Thus they receive opposite T/F colors.
- For each clause $(\ell_1 \vee \ell_2 \vee \ell_3)$, attach the standard clause gadget to its three literal vertices. The gadget is 3-colorable exactly when at least one literal has color T .

$\Rightarrow$. If the formula is satisfiable, color each literal vertex from its truth value. Every clause has a true literal, so every clause gadget can be 3-colored. Hence the whole graph is 3-colorable.

$\Leftarrow$. If the graph has a 3-coloring, each pair $x_i, \neg x_i$ gets opposite T/F colors, giving a truth assignment. Every clause gadget forces at least one of its literals to be T , so every clause is satisfied.

Time. Each variable and clause adds only a constant-size gadget, so the graph size is polynomial.

Conclusion. 3-COLOR is NP-complete. Since it is a special case of VERTEX-COLOR,

VERTEX-COLOR is NP-complete.

What to Draw if Asked

Draw the T, F, B triangle, the variable pair $x_i, \neg x_i$, and your course's standard 3-SAT clause gadget.

3 Clique Partition (Graph Partition) is NP-Complete

Problem

Can $V(G)$ be partitioned into at most k cliques?

Reduction

VERTEX-COLOR $\leq_p$ CLIQUE-PARTITION

Given (G, k) , output $(\overline{G}, k)$.

Actual Proof

In NP. Certificate = a partition into at most k groups. Check that every pair inside each group is adjacent.

Build. Construct the complement graph $\overline{G}$.

$\Rightarrow$. If G has a proper k -coloring, each color class is an independent set in G . Therefore each color class becomes a clique in $\overline{G}$. So $\overline{G}$ can be partitioned into at most k cliques.

$\Leftarrow$. If $\overline{G}$ is partitioned into at most k cliques, each part is an independent set in G . Give one color to each part. No adjacent vertices of G receive the same color, so G is k -colorable.

Time. Constructing $\overline{G}$ takes $O(|V|^2)$.

Conclusion.

CLIQUE-PARTITION is NP-complete.

4 Edge Coloring is NP-Complete

Problem

Can the edges of G be colored with at most k colors so that edges sharing a vertex have different colors?

Known NP-Complete Source

Holyer's theorem: deciding whether a *cubic* graph is 3-edge-colorable is NP-complete.

Actual Proof

In NP. Certificate = one color per edge. For each vertex, check that all incident edges have different colors.

Build. Take an instance G of cubic 3-edge-coloring and output the general edge-coloring instance $(G, 3)$.

$\Rightarrow$. If the cubic graph G is 3-edge-colorable, then $(G, 3)$ is clearly a YES-instance of EDGE-COLOR.

$\Leftarrow$. If $(G, 3)$ is a YES-instance of EDGE-COLOR, then the same coloring is a valid 3-edge-coloring of the original cubic graph.

Time. The reduction only copies G and writes $k = 3$, so it is polynomial.

Conclusion. By Holyer's NP-completeness result and membership in NP,

EDGE-COLOR is NP-complete.

5 Graph Isomorphism: In NP, Not Known NP-Complete

Problem

Are G_1 and G_2 the same graph up to a renaming of vertices?

Proof that Graph Isomorphism is in NP

Certificate. A bijection $f : V(G_1) \rightarrow V(G_2)$.

Verify. Check that f is one-to-one and onto. Then for every pair u, v , check

$$(u, v) \in E(G_1) \iff (f(u), f(v)) \in E(G_2).$$

There are at most $O(|V|^2)$ pairs, so verification is polynomial.

Conclusion.

GRAPH-ISOMORPHISM $\in$ NP.

Important

Ordinary GRAPH-ISOMORPHISM is **not known to be NP-complete**. Do not write an NP-completeness proof for it unless the question is actually about SUBGRAPH-ISOMORPHISM.

6 Subgraph Isomorphism is NP-Complete

Problem

Is G_1 isomorphic to some subgraph of G_2 ?

Reduction

$$\text{CLIQUE} \leq_p \text{SUBGRAPH-ISOMORPHISM}$$

Given (G, k) , set $G_1 = K_k$ and $G_2 = G$.

Actual Proof

In NP. Certificate = an injective mapping from vertices of G_1 to vertices of G_2 . Check that every edge of G_1 maps to an edge of G_2 .

Build. From (G, k) , construct the complete graph K_k and output (K_k, G) .

$\Rightarrow$. If G has a clique of size k , the subgraph induced by those k vertices contains a copy of K_k . Hence K_k is isomorphic to a subgraph of G .

$\Leftarrow$. If K_k is isomorphic to a subgraph of G , the corresponding k vertices of G are pairwise adjacent. Therefore they form a clique of size k .

Time. Building K_k takes $O(k^2)$, so the reduction is polynomial.

Conclusion.

$$\text{SUBGRAPH-ISOMORPHISM is NP-complete.}$$

Graph Reductions

Compact Actual Proofs - Exam Ready

Proof Format

For each reduction, write only:

$$\boxed{\text{Build}} \rightarrow \boxed{\Rightarrow} \rightarrow \boxed{\Leftarrow} \rightarrow \boxed{\text{Time}} \rightarrow \boxed{\text{Result}}$$

The important line is always

$$\boxed{\text{source is YES} \iff \text{target is YES.}}$$

Reductions to Memorize

$$\text{VERTEX-COVER} \leftrightarrow_p \text{INDEPENDENT-SET}, \quad \text{INDEPENDENT-SET} \leftrightarrow_p \text{CLIQUE}$$

$$\text{VERTEX-COVER} \leq_p \text{SET-COVER}, \quad 3\text{-SAT} \leq_p \text{INDEPENDENT-SET}.$$

For the Set-Cover pair, the standard direct proof is $\text{VERTEX-COVER} \leq_p \text{SET-COVER}$.

1 Decision Problems at a Glance

Definitions

Problem	YES condition
VERTEX-COVER(G, k)	There is a set $C \subseteq V$ with $ C \leq k$ touching every edge.
INDEPENDENT-SET(G, k)	There is a set $S \subseteq V$ with $ S \geq k$ and no edge joins two vertices of S .
CLIQUE(G, k)	There is a set $Q \subseteq V$ with $ Q \geq k$ and every pair in Q is adjacent.
SET-COVER($U, \mathcal{S}, k$)	At most k sets from $\mathcal{S}$ cover all elements of U .
3-SAT(ϕ)	Some truth assignment satisfies every 3-literal clause of ϕ .

Two Lemmas That Do Most of the Work

$$\boxed{C \text{ is a vertex cover in } G \iff V - C \text{ is independent in } G}$$

$$\boxed{S \text{ is independent in } G \iff S \text{ is a clique in } \overline{G}}$$

2 Vertex Cover $\leftrightarrow$ Independent Set

Reduction

For $n = |V(G)|$:

$$\boxed{(G, k) \mapsto (G, n - k)}$$

The graph stays the same; only the parameter changes.

Actual Proof: VERTEX-COVER $\leq_p$ INDEPENDENT-SET**Build.** From VERTEX-COVER(G, k) output INDEPENDENT-SET($G, n - k$). $\Rightarrow$. Suppose C is a vertex cover with $|C| \leq k$. If two vertices outside C were adjacent, that edge would be uncovered. Hence $V - C$ is independent, and

$$|V - C| = n - |C| \geq n - k.$$

So the new INDEPENDENT-SET instance is YES.

 $\Leftarrow$. Suppose S is independent with $|S| \geq n - k$. No edge can have both endpoints in S , so every edge touches $V - S$. Thus $V - S$ is a vertex cover, and

$$|V - S| = n - |S| \leq k.$$

So the original VERTEX-COVER instance is YES.

Time. Only $n - k$ is computed, so the reduction is polynomial.**Result.**

$$(G, k) \in \text{VERTEX-COVER} \iff (G, n - k) \in \text{INDEPENDENT-SET}.$$

Reverse: INDEPENDENT-SET $\leq_p$ VERTEX-COVERUse the same transformation $(G, k) \mapsto (G, n - k)$.If S is independent with $|S| \geq k$, then $V - S$ is a vertex cover of size

$$|V - S| \leq n - k.$$

If C is a vertex cover with $|C| \leq n - k$, then $V - C$ is independent and

$$|V - C| \geq k.$$

Therefore

$$(G, k) \in \text{INDEPENDENT-SET} \iff (G, n - k) \in \text{VERTEX-COVER}.$$

Tiny ExampleFor the path $a - b - c$, $\{b\}$ is a vertex cover of size 1 and its complement $\{a, c\}$ is an independent set of size $3 - 1 = 2$.

3 Independent Set $\leftrightarrow$ Clique

ReductionConstruct the complement graph $\overline{G}$:

$$(u, v) \in E(\overline{G}) \iff u \neq v \text{ and } (u, v) \notin E(G).$$

Then use

$$(G, k) \mapsto (\overline{G}, k).$$

Actual Proof: INDEPENDENT-SET $\leq_p$ CLIQUE**Build.** From INDEPENDENT-SET(G, k) output CLIQUE($\overline{G}, k$). $\Rightarrow$. Let S be independent in G . No pair in S is an edge of G , so every pair becomes an edge of $\overline{G}$. Hence S is a clique in $\overline{G}$. $\Leftarrow$. Let S be a clique in $\overline{G}$. Every pair in S is an edge of $\overline{G}$, so none of those pairs is an edge of G . Hence S is independent in G .

Time. $\overline{G}$ is constructed by checking vertex pairs, in $O(|V|^2)$ time.

Result.

$$(G, k) \in \text{INDEPENDENT-SET} \iff (\overline{G}, k) \in \text{CLIQUE}.$$

Reverse: $\text{CLIQUE} \leq_p \text{INDEPENDENT-SET}$

Use the same map $(G, k) \mapsto (\overline{G}, k)$. A clique in G is exactly an independent set in $\overline{G}$. Therefore

$$(G, k) \in \text{CLIQUE} \iff (\overline{G}, k) \in \text{INDEPENDENT-SET}.$$

The construction is polynomial.

4 Vertex Cover $\leq_p$ Set Cover

Construction

Given $\text{VERTEX-COVER}(G = (V, E), k)$, create:

$$U = E$$

and for every vertex $v \in V$ create

$$S_v = \{e \in E : e \text{ is incident on } v\}.$$

Let $\mathcal{S} = \{S_v : v \in V\}$ and keep the same k .

Actual Proof

$\Rightarrow$. Suppose C is a vertex cover with $|C| \leq k$. Choose the sets S_v for all $v \in C$. Every edge has an endpoint in C , so every element of $U = E$ appears in at least one chosen set. Thus these at most k sets cover U .

$\Leftarrow$. Suppose at most k sets $S_{v_1}, \dots, S_{v_r}$ cover $U = E$. Let

$$C = \{v_1, \dots, v_r\}.$$

Every edge e belongs to some chosen S_{v_i} , which means e is incident on v_i . Hence every edge is touched by C , so C is a vertex cover and $|C| = r \leq k$.

Time. We create one set per vertex and list its incident edges; this is polynomial in $|V| + |E|$.

Result.

$$(G, k) \in \text{VERTEX-COVER} \iff (U, \mathcal{S}, k) \in \text{SET-COVER}.$$

Tiny Example

If $E = \{ab, bc\}$, then

$$U = \{ab, bc\}, \quad S_a = \{ab\}, \quad S_b = \{ab, bc\}, \quad S_c = \{bc\}.$$

Choosing vertex b is the same as choosing set S_b .

5 3-SAT $\leq_p$ Independent Set

Construction

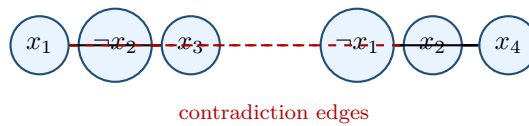
Let

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m.$$

Build graph G :

- one vertex for each literal occurrence;
- the 3 vertices of each clause form a triangle;
- connect complementary literals from different clauses;
- set $k = m$.

$$C_1 = (x_1 \vee \neg x_2 \vee x_3) \quad C_2 = (\neg x_1 \vee x_2 \vee x_4)$$



Actual Proof

$\Rightarrow$. Suppose ϕ is satisfiable. From each clause, choose one literal that is true. This gives exactly m vertices.

No two chosen vertices are adjacent:

- only one was chosen from each clause triangle;
- two true literals under one assignment cannot be complementary.

Therefore the chosen vertices form an independent set of size m .

$\Leftarrow$. Suppose G has an independent set S of size m . A clause triangle contributes at most one vertex. Since there are m triangles, S must contain exactly one vertex from each clause. Also, S cannot contain complementary literals because those vertices are connected.

Set every selected literal to true. The choices are consistent, and every clause contains one selected true literal. Hence ϕ is satisfiable.

Time. There are $3m$ vertices and at most polynomially many edges.

Result.

$$\phi \in \text{3-SAT} \iff (G, m) \in \text{INDEPENDENT-SET}.$$

6 Final Memory Sheet

One-Line Proof Triggers

Reduction		Remember this
VERTEX-COVER	$\leftrightarrow$	complement of a vertex cover is an independent set; use $n - k$.
INDEPENDENT-SET		
INDEPENDENT-SET	$\leftrightarrow$	complement the graph; keep k .
CLIQUE		
VERTEX-COVER	$\leq_p$	universe = edges; one set per vertex.
SET-COVER		
3-SAT	$\leq_p$	one triangle per clause; contradiction edges; target $k = m$.
INDEPENDENT-SET		